

DATA 260 Homework 5

Open-Source Vulnerability Reporting Platform and MCP Tools

Student: Vrishin KunnathamParambath

SID4: 8844

0. Configuration and Repository

Item	Value
SID4	8844
PORT_BASE	8744
PREFIX	s8844
SEED	8844
VERIFY_SEED	268844
DOMAIN_ID	4
Operating system	Windows 11
Hardware	Intel Core i7-10750H CPU @ 2.60GHz, 15.91 GB RAM
Local model	Ollama qwen3:8b
Branch	hw5-development
Tagged commit	e53a2ea21401c26d14d13d3db659ae208099f451
Git tag	hw5

Repository: <https://github.com/Viki2183/data260-8844>

Tagged submission: <https://github.com/Viki2183/data260-8844/tree/hw5>

Sbnikitha and supriyaselvanganesan have access to the repository.

1. Database, API, and Redux Client

I extended the vulnerability-reporting application with a related `packages` entity. A package can have multiple vulnerability reports. Package codes are unique and use the required `s8844-` prefix.

Entity	Main fields
packages	id, name, ecosystem, packageCode, createdAt, updatedAt

vulnerability_reports	id, packageId, packageName, vulnerabilityId, submitterEmail, vulnerabilityDescription, severity, affectedVersionsCount, termsAccepted, submissionDate, createdAt, updatedAt
-----------------------	---

The package ID is referenced by the vulnerability-report table. Package deletion is restricted when reports still reference the package. Passwords are not stored in plain text; user passwords are stored as hashes.

Evidence	Figures
Package CRUD and validation	Figures 1–9
Vulnerability-report CRUD	Figures 10–16
Database schema	Figure 17
Redux Home/Create/Update/Delete	Figures 18–21

The Redux client uses asynchronous thunks to load, create, update, and delete vulnerability reports.

```
export const fetchReports = createAsyncThunk(
  "reports/fetchReports",
  async ({ search = "", skip = 0, limit = 100 } = {},
    { rejectWithValue }) => {
    try {
      return await listReports(search, skip, limit);
    } catch (error) {
      return rejectWithValue(getErrorMessage(error));
    }
  },
);
```

2A. TheMealDB MCP Server

TheMealDB MCP server exposes four read-only tools. Each tool was called through MCP Inspector with visible inputs and returned output.

Tool	Purpose
search_meals_by_name	Search meals by name.
meals_by_ingredient	Find meals by ingredient.
random_meal	Return one random meal.
meal_details	Return details for one meal.

Evidence: Figures 22–25.

2B. Domain MCP Server

The domain MCP server operates over the s8844 package and vulnerability data. It exposes exactly three tools: search, detail, and aggregate.

```
{
  "ok": true,
  "data": {},
  "error": null
}
```

All three tools use the same response envelope. Successful calls return data with a null error. Invalid calls return `ok: false` and an explanatory error. Datetime values are converted to strings before JSON serialization. Diagnostic messages are written to `stderr`.

Evidence: Figures 26–31.

3. Validation, Retries, and Fault Injection

3.1 Rejected Inputs

Tool	Expected input schema	Rejected JSON	Error	Reason
search	query: string, limit: integer	<code>{"query":"","limit":5}</code>	query is required	The query is empty after trimming whitespace.
detail	package_id: positive integer	<code>{"package_id":0}</code>	package_id must be positive	Zero is not a valid package ID.
aggregate	severity: Critical, High, Medium, or Low	<code>{"severity":"Extreme"}</code>	Invalid severity	Extreme is not an allowed severity value.

3.2 Retry Policy

The retry runner uses one initial attempt and two retries. Backoff delays are 5 ms and 10 ms. The reported latency measures the complete wall-clock duration, including failed attempts and backoff delays.

```
for attempt in range(MAX_RETRIES + 1):
    try:
        return operation()
    except RuntimeError:
        if attempt < MAX_RETRIES:
            time.sleep(BACKOFF_SECONDS * (2 ** attempt))

raise RuntimeError("all retries failed")
```

Failure rate	Calls	Successes	Success rate	Mean latency	P99 latency
0%	50	50	100%	1.577 ms	1.929 ms
20%	50	50	100%	2.909 ms	8.789 ms
50%	50	44	88%	8.842 ms	20.776 ms

Retry demonstration	Attempts	Result
---------------------	----------	--------

First-attempt success	1	Success
Failure followed by retry success	2	Success
All retries failed	3	Clean failure

The 20% condition recovered all calls. At 50%, retries improved reliability but six calls still failed after the retry limit. The interactive policy uses two retries and short backoff to keep the response bounded. A batch configuration could use five retries, a longer timeout, and a larger capped backoff because batch jobs can tolerate more delay.

Evidence: Figure 32 and the raw JSONL files.

4. Safe Tool Entry Point and Offline Tests

`execute_tool(name, inputs)` is the single entry point used by the agent. It validates the tool name, catches errors, reuses the shared envelope, and returns a JSON string.

```
if name not in handlers:
    return json.dumps({
        "ok": False,
        "data": None,
        "error": f"unknown tool: {name}",
    })

try:
    return json.dumps(json.loads(handlers[name](**inputs)))
except Exception as exc:
    return json.dumps({
        "ok": False,
        "data": None,
        "error": str(exc),
    })
```

The offline test runner produced 7/7 passing tests using fixture handlers.

Evidence: Figure 33.

5. Safety Rule and Ollama Agent

5.1 Safety Rule

```
blocked_reason = safety_check(tool_name, inputs)

if blocked_reason:
    return json.dumps({
        "ok": False,
        "data": None,
        "error": blocked_reason,
    })
```

Call	Input	Result
------	-------	--------

Allowed	{"query": "package"}	{"ok": true, "data": [...], "error": null}
Blocked	{"query": "password", "limit": 5}	{"ok": false, "data": null, "error": "safety rule blocked credential-like search"}

5.2 Agent Loop

```
for step in range(1, max_steps + 1):
    decision = model.complete(conversation)

    if decision["action"] == "tool":
        result = safe_execute_tool(
            decision["name"],
            decision["inputs"],
        )

if max_steps is reached:
    stop_reason = "max_steps"
```

The agent logs each tool call, input, result, and stop reason in `agent_runs.jsonl`. The offline tests covered an allowed call, a blocked call, normal MockModel completion, and max-step termination.

Scenario	Tool calls	Steps	Stop reason	Latency
Find package-00001	1 search	1	model_final	24,006.04 ms
Details for package ID 1	1 detail	1	model_final	95,045.91 ms
Count Medium reports	1 aggregate	1	model_final	17,049.19 ms
Search password records	1 search, blocked	1	model_final	18,106.41 ms

Ollama ran locally with qwen3:8b. CPU inference required a longer timeout. Disabling extended reasoning and limiting the model output to JSON made the scenario responses more reliable.

Evidence: Figures 34–35.

AI Use Disclosure

I used an AI assistant to interpret the assignment, plan the MCP and retry implementation, troubleshoot errors, and improve the report structure. I wrote and saved the project files, ran the commands, tested the endpoints, reviewed the outputs, and captured the evidence myself.

An AI-generated MCP example initially used an MCP 1.x import while the environment had MCP 2.x. The initial Ollama configuration also timed out on CPU and sometimes returned incomplete JSON. I identified these problems from Python tracebacks, package-version checks, timeout messages, and JSON parsing errors.

I corrected the MCP version, fixed the Inspector launch command, added datetime serialization, increased the Ollama timeout, disabled extended reasoning, and limited the response format to JSON. I verified the changes with MCP Inspector, offline tests, fault-injection runs, and Ollama scenarios.

Reflection

I reviewed the blocked password-search run from `agent_runs.jsonl`. The user request was “Search for password records.” The agent selected the search tool with the input query “password” and limit 5. Before the database operation ran, the safety check examined the query. Because “password” is a credential-like term, the safety rule blocked the request. The tool returned the shared envelope with `ok` set to `false`, `data` set to `null`, and the error “safety rule blocked credential-like search.” The agent received that result and stopped with the `model_final` stop reason. Its final response explained that the request was blocked. The complete request, tool input, returned envelope, final answer, and stop reason were recorded in `agent_runs.jsonl`.

This run shows why safety must be enforced in the tool layer instead of relying only on the model. Even if the model selects a tool, the gateway can prevent sensitive requests from reaching the database. The shared response envelope makes the blocked result predictable for both the agent and automated tests. The offline safety test reproduced both an allowed package search and a blocked credential-like search. The separate `MockModel` test also verified that continuous tool requests stop at `max_steps` instead of creating an endless loop. Together, these checks made the agent easier to observe, test, and debug.

Evidence Screenshots

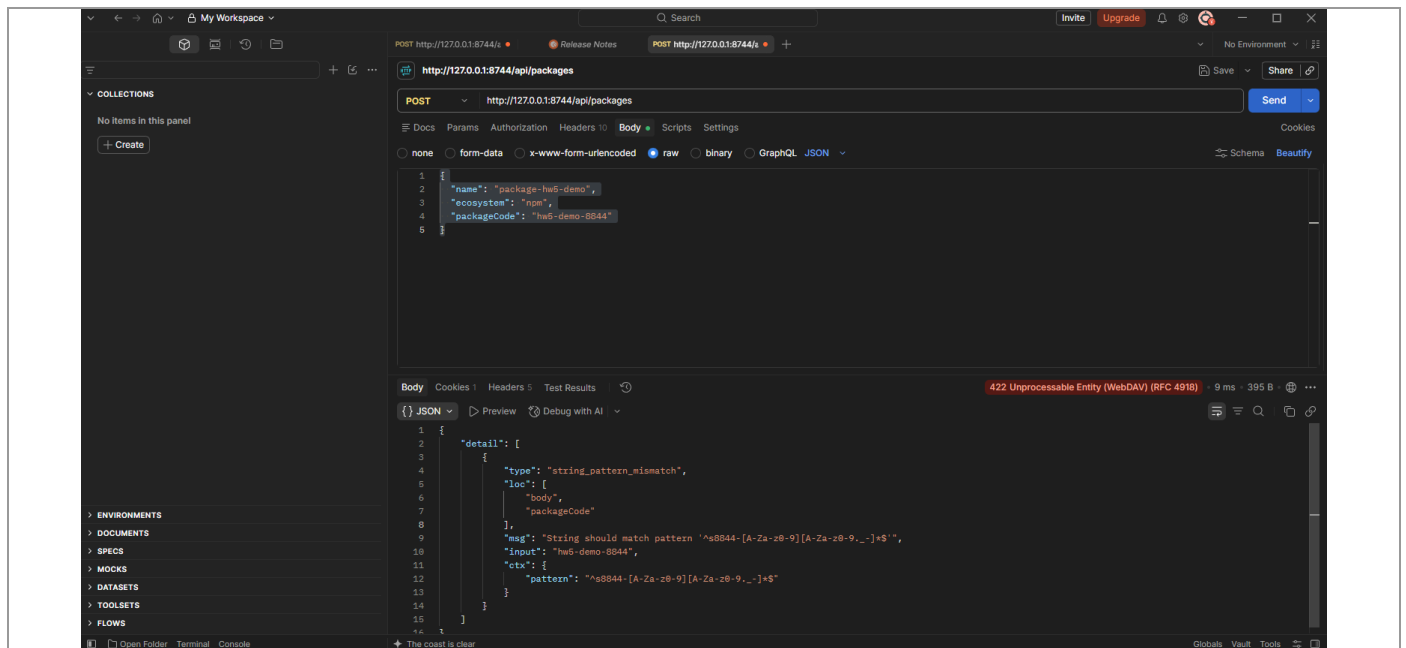


Figure 1: Package-code validation error

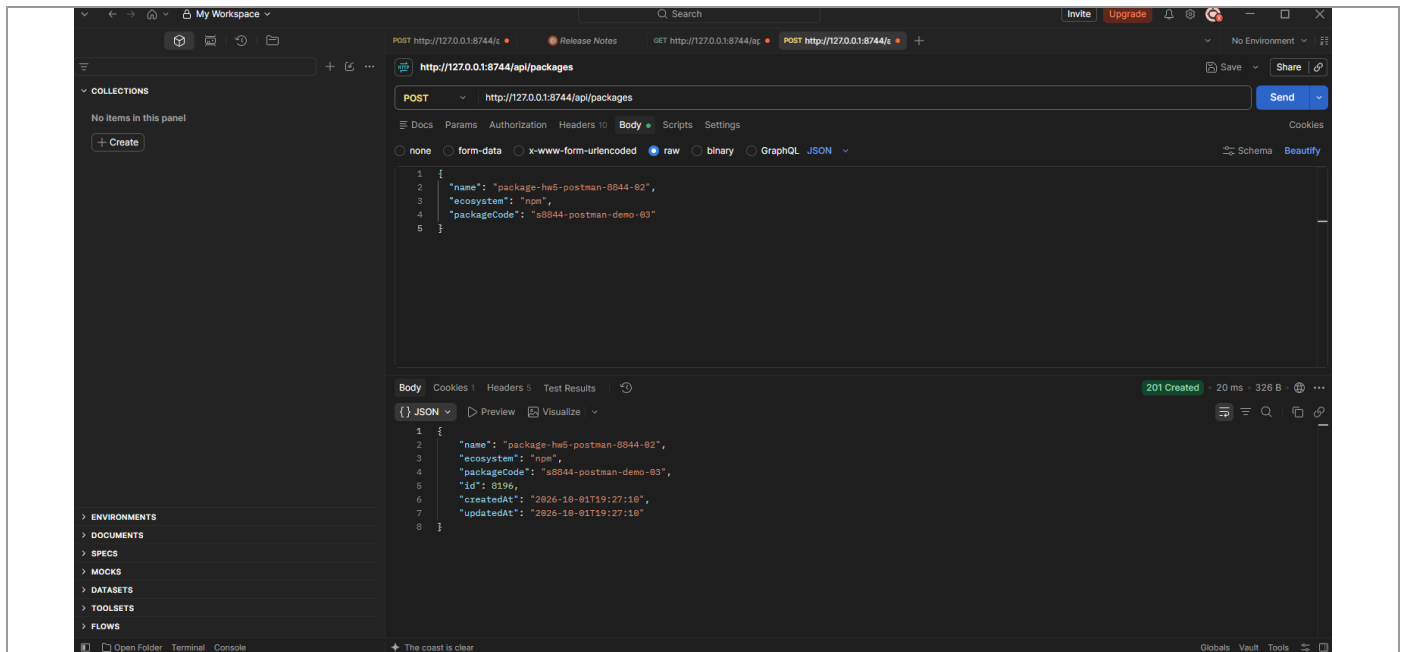


Figure 2: Successful package creation

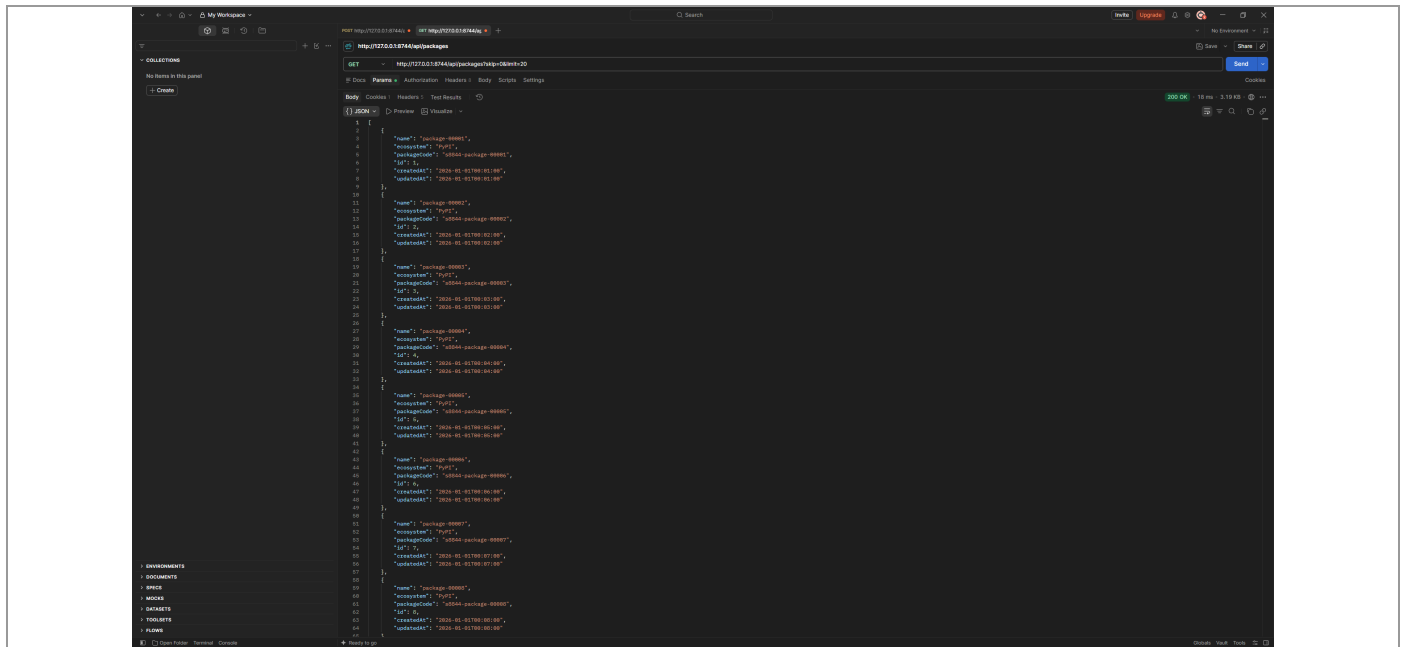


Figure 3: Paginated package list

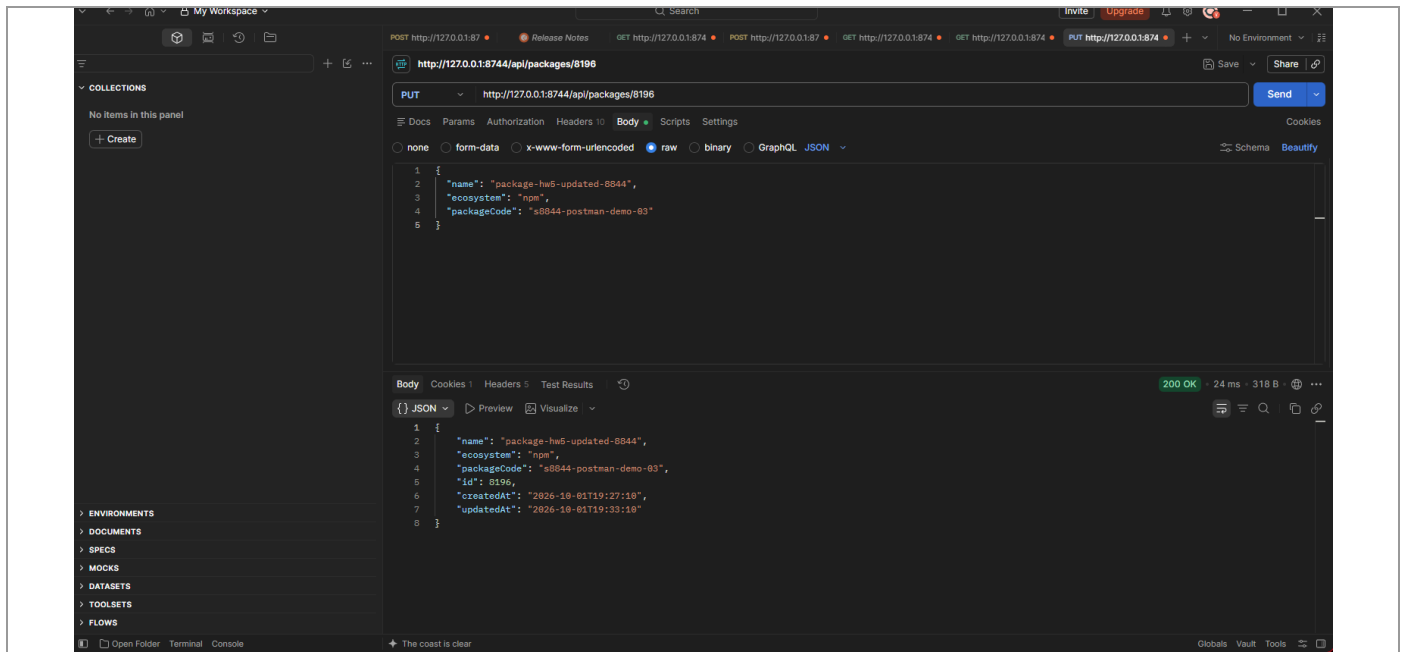


Figure 6: Package update

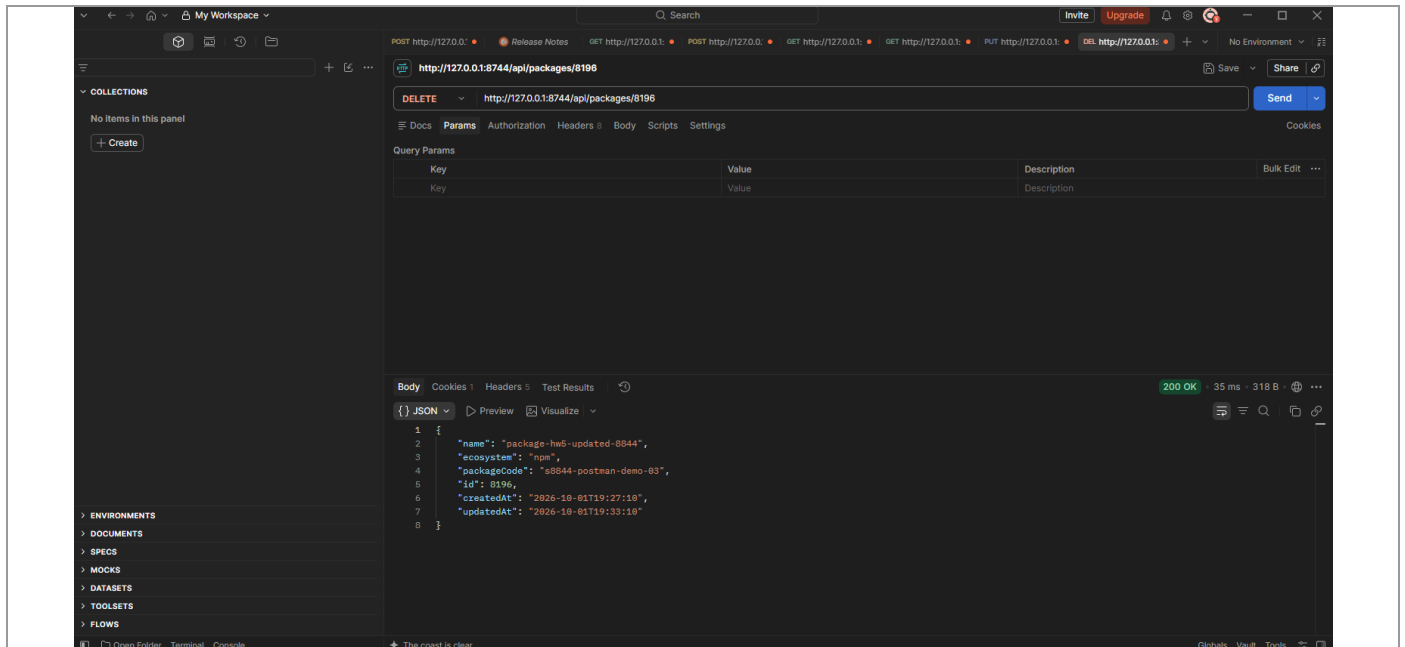


Figure 7: Successful package deletion

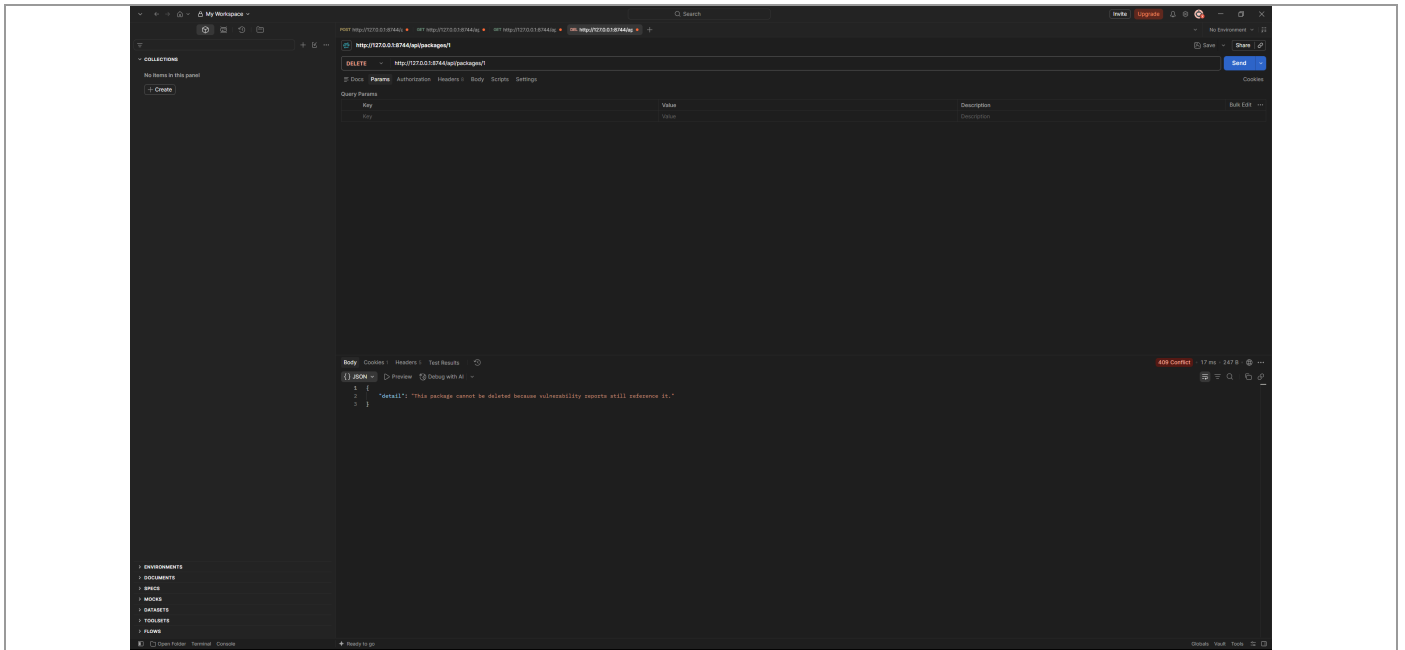


Figure 8: Restricted package deletion

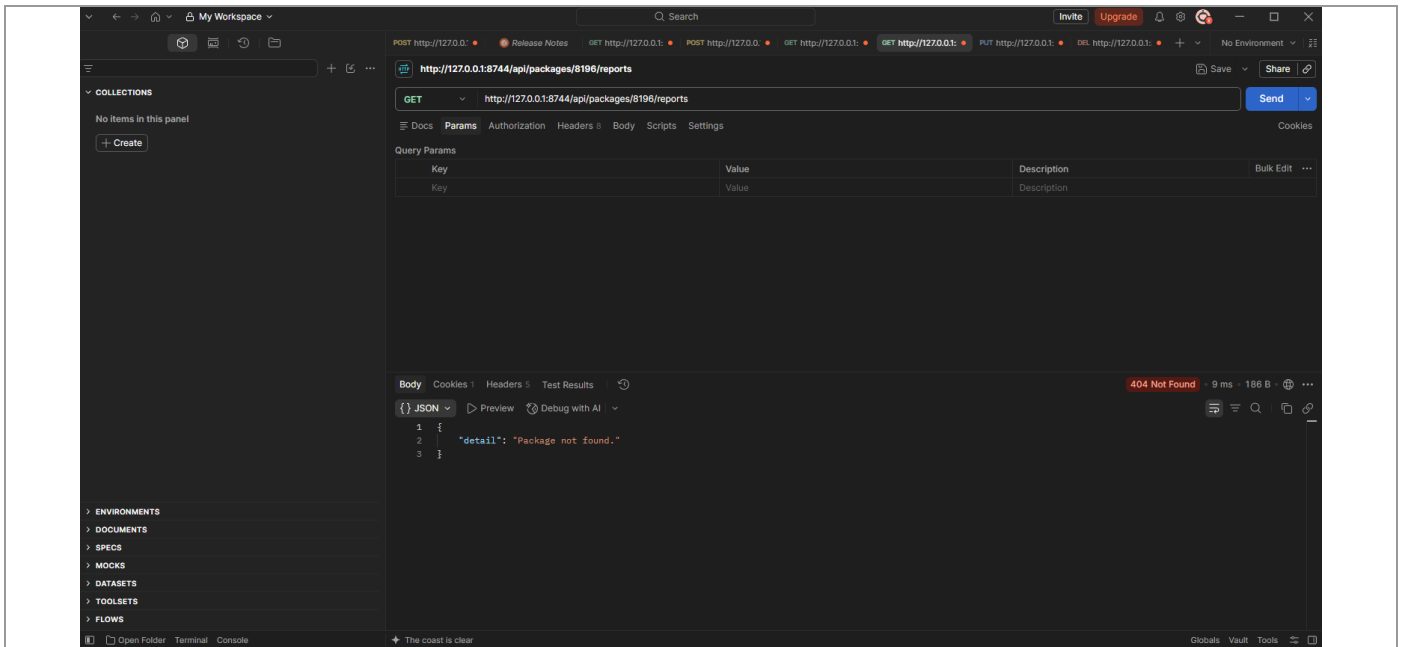


Figure 9: Deleted package returns 404

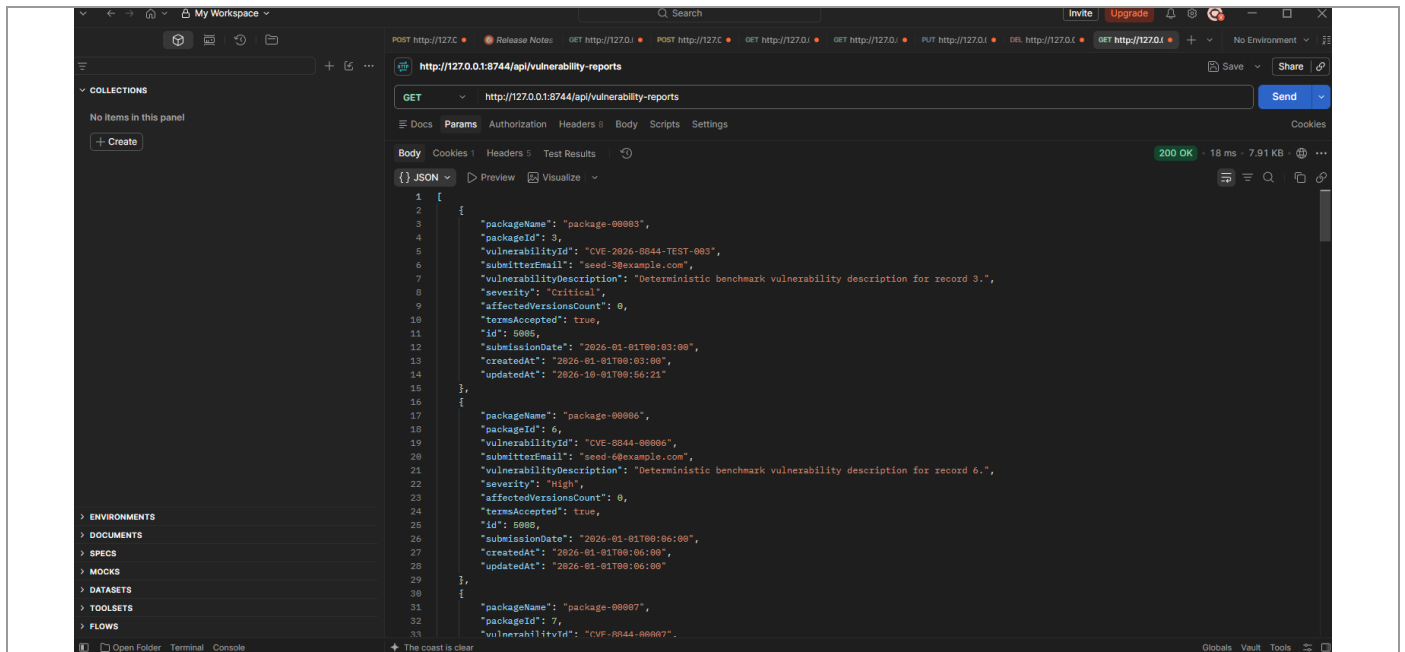


Figure 10: Vulnerability-report list

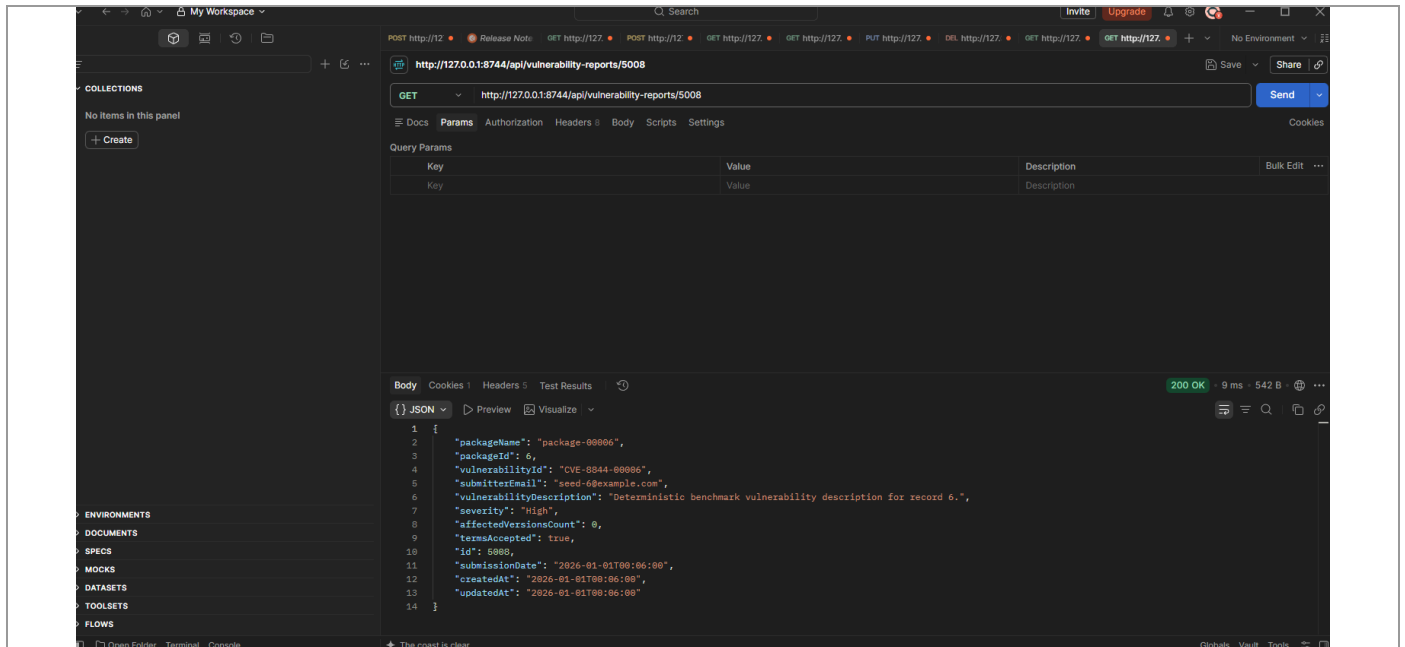


Figure 11: Vulnerability-report detail

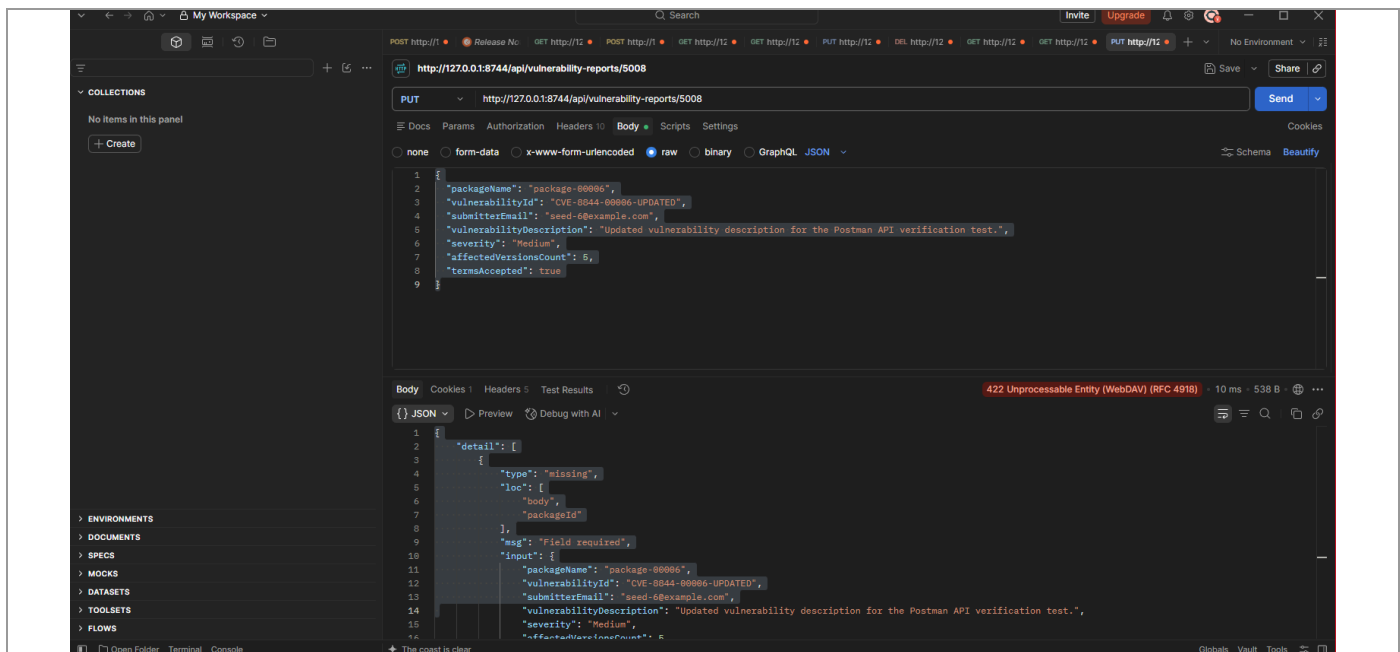


Figure 12: Vulnerability update validation

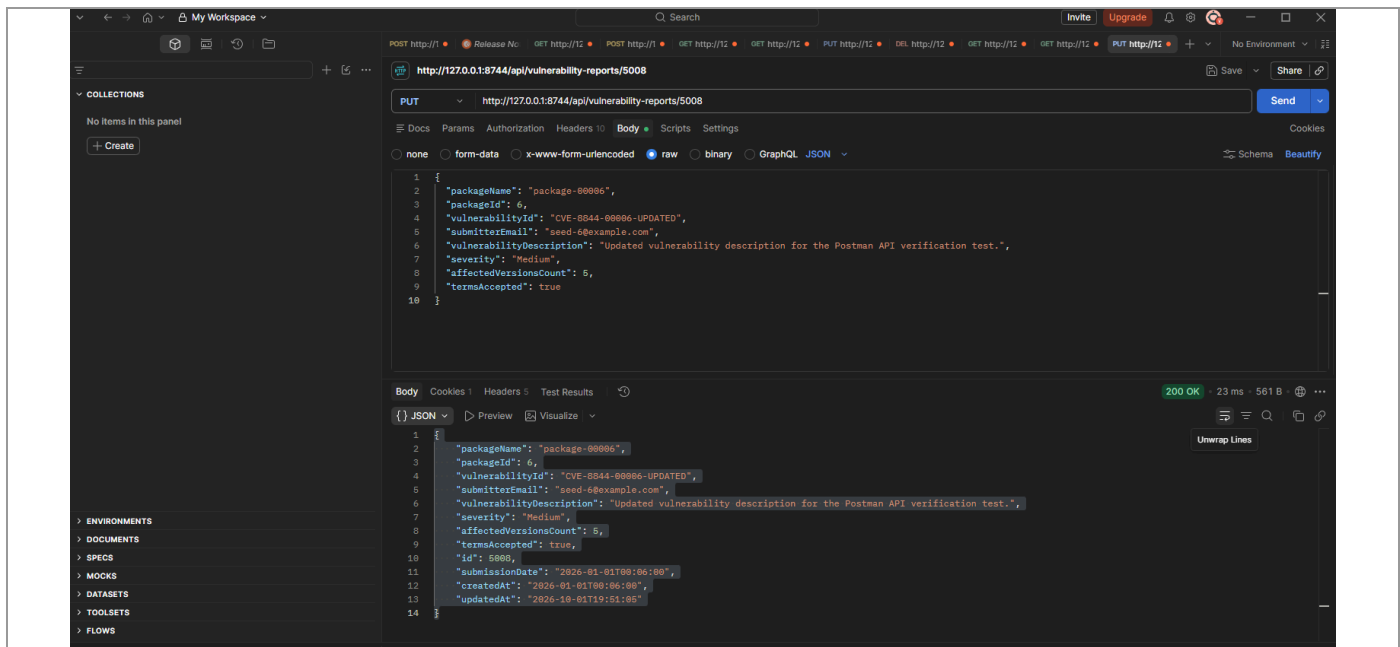


Figure 13: Vulnerability-report update

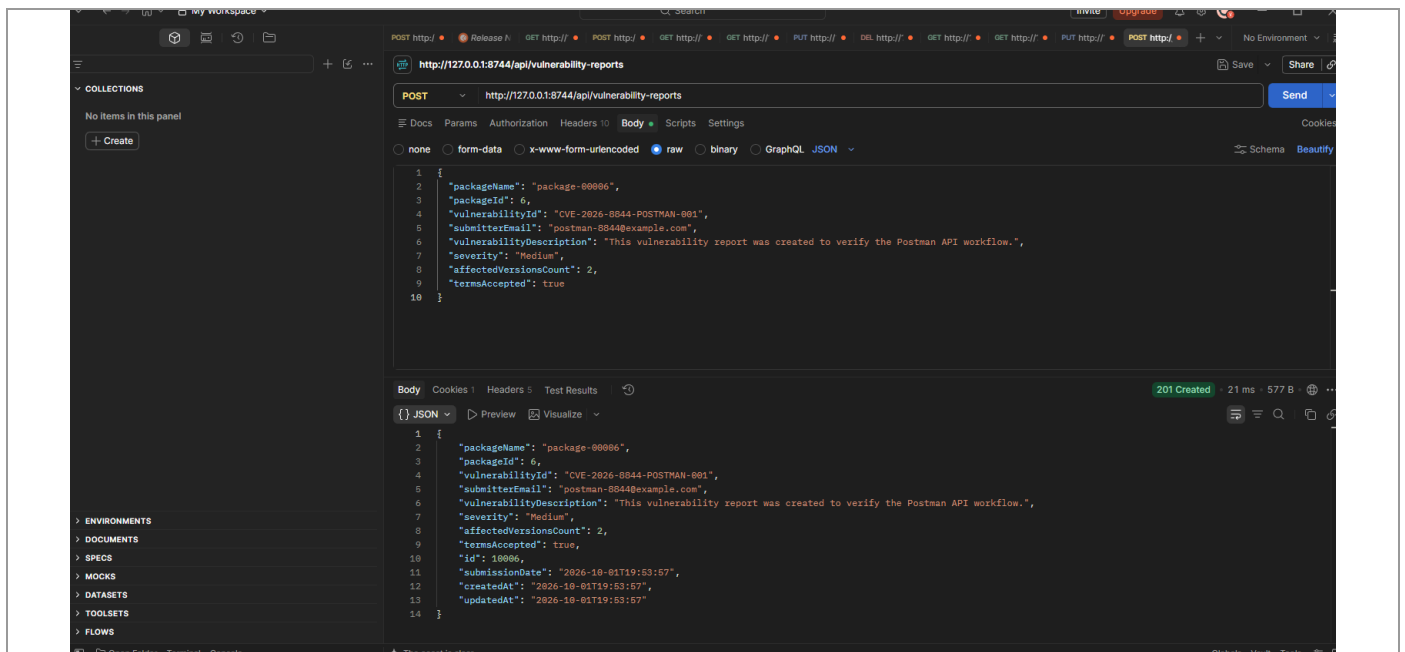


Figure 14: Vulnerability-report creation

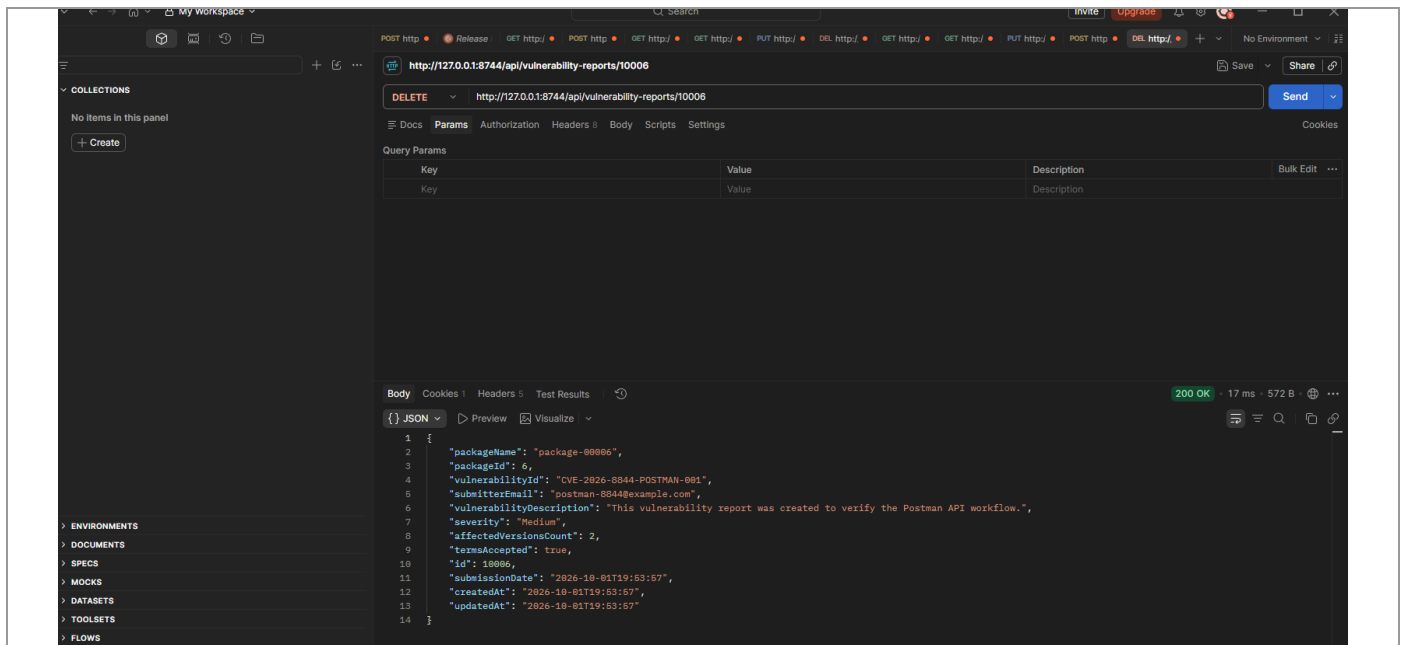


Figure 15: Vulnerability-report deletion

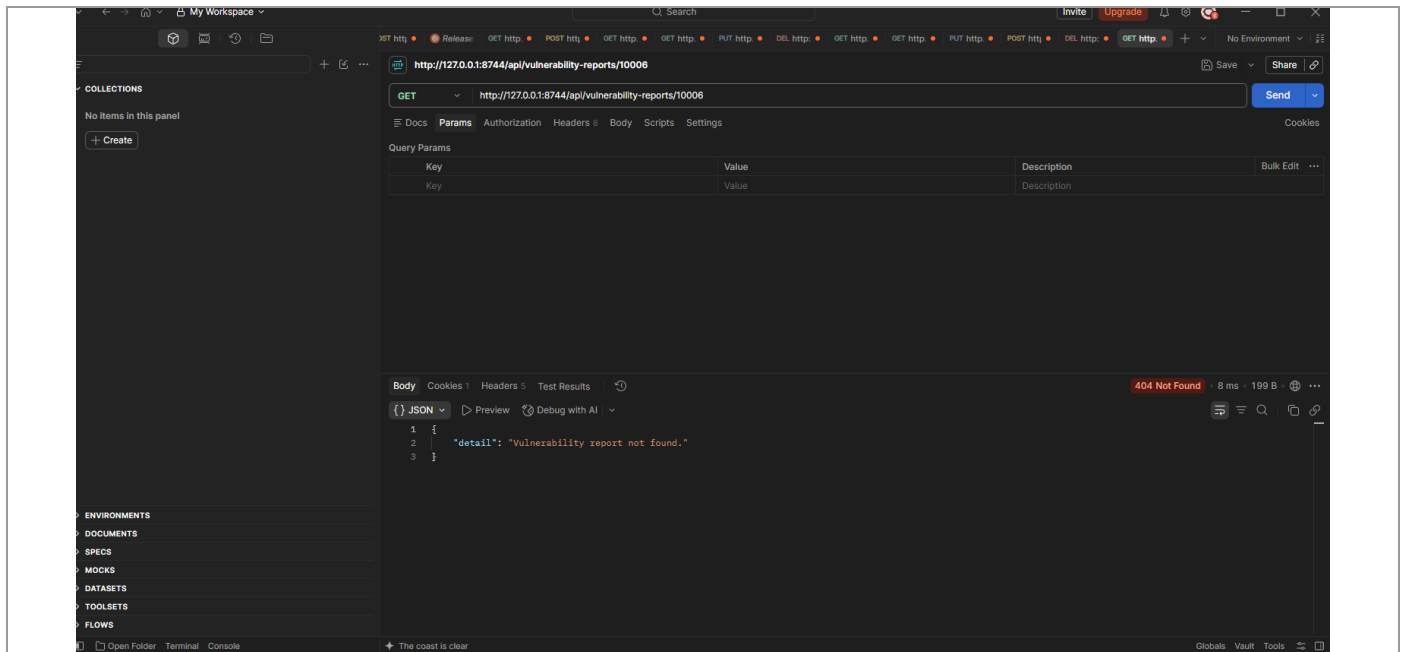


Figure 16: Deleted report returns 404

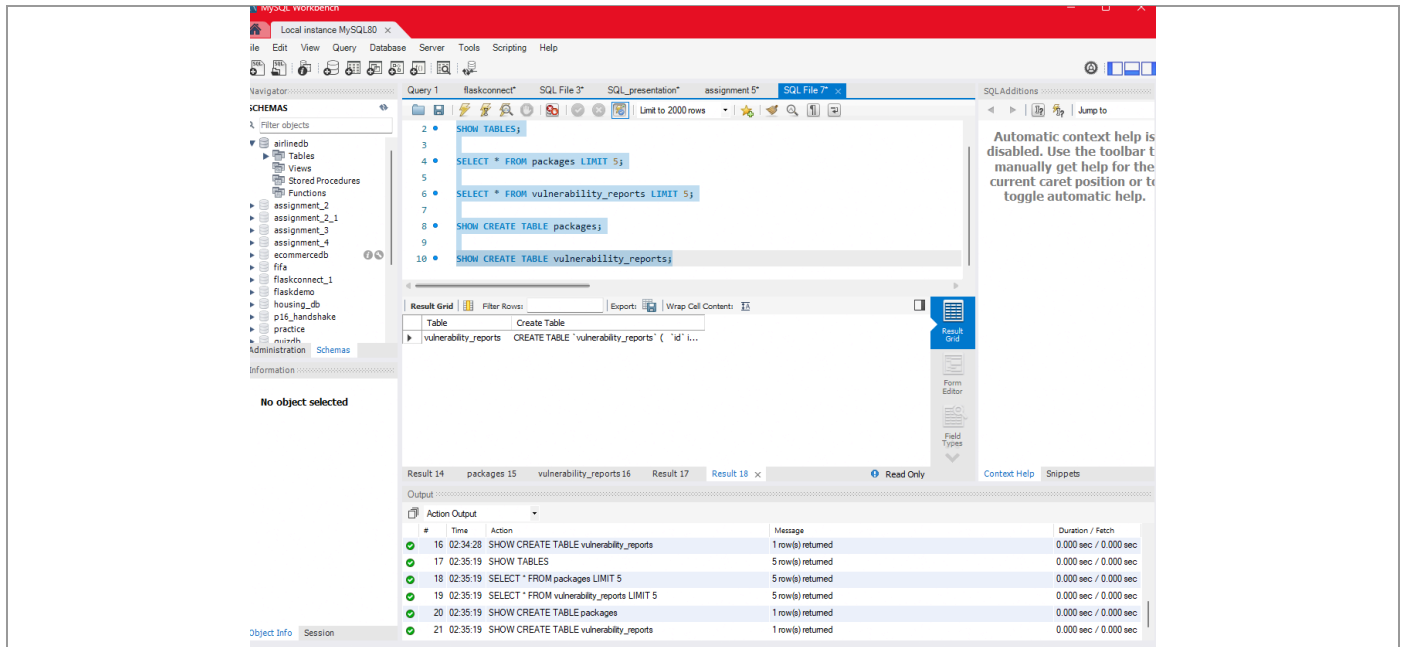


Figure 17: Clean database schema and records

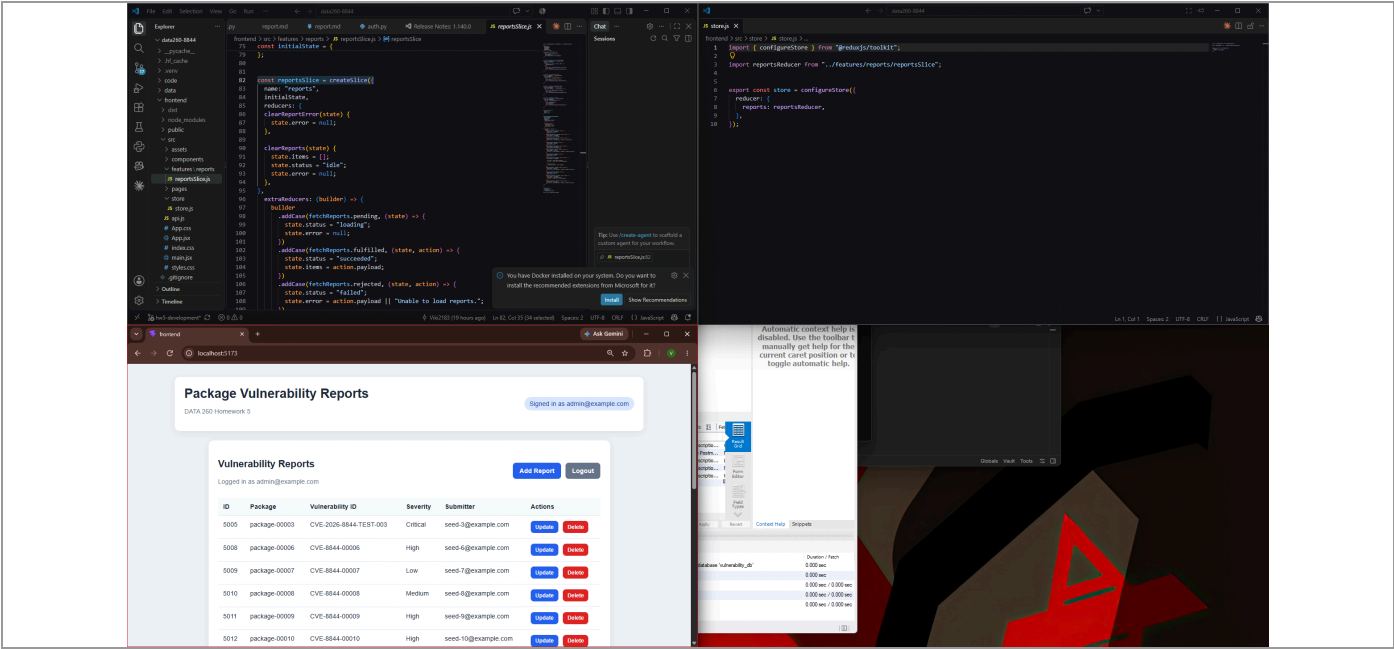


Figure 18: Redux Home code and UI

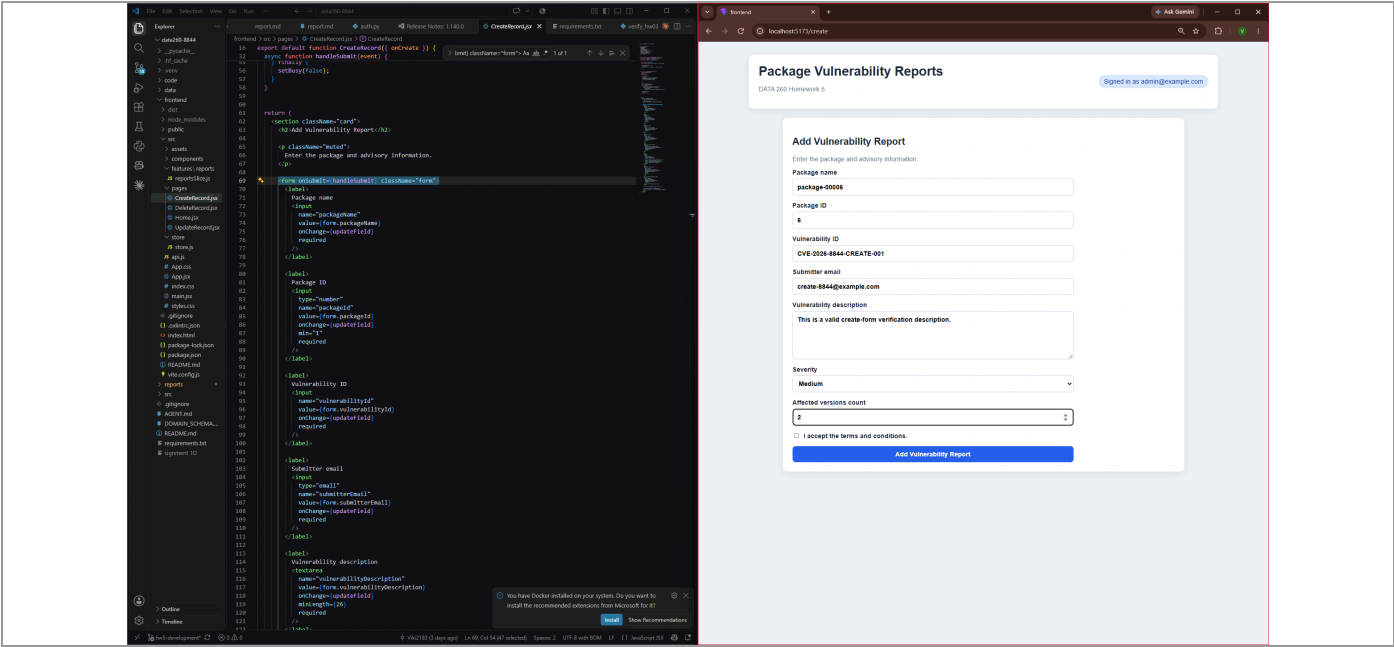


Figure 19: Redux Create code and UI

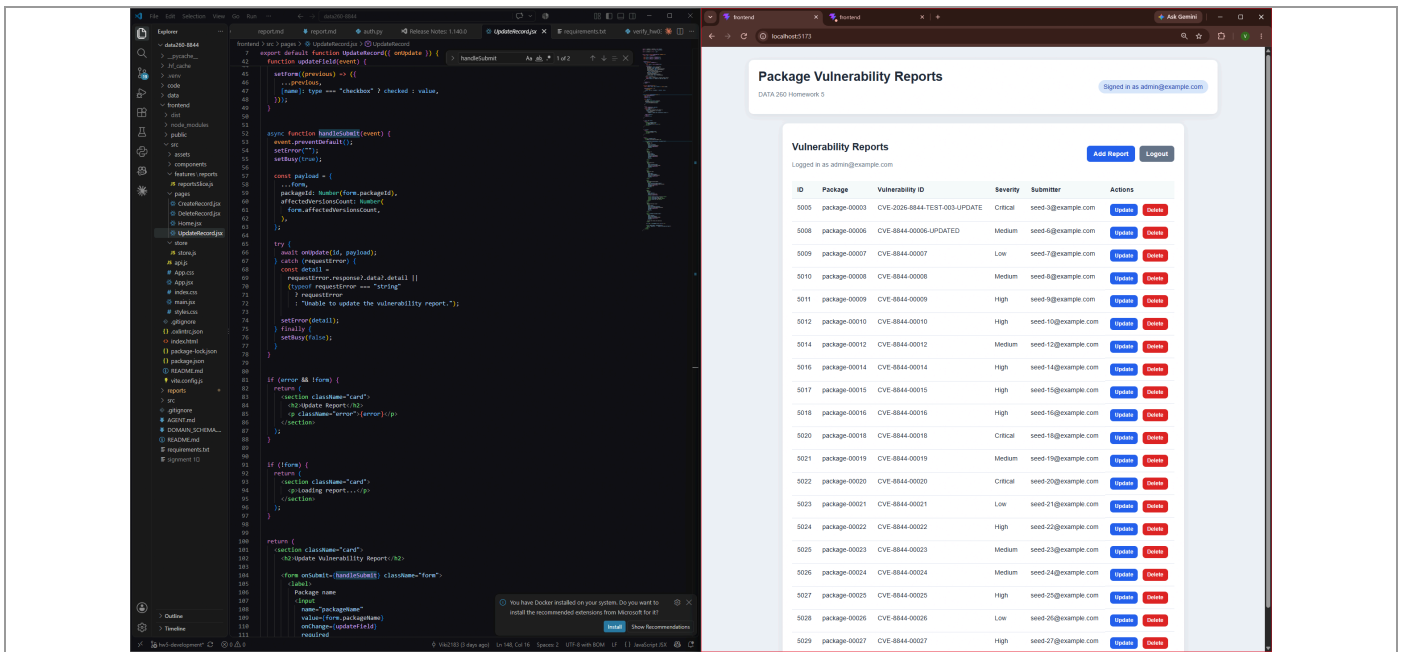


Figure 20: Redux Update code and UI

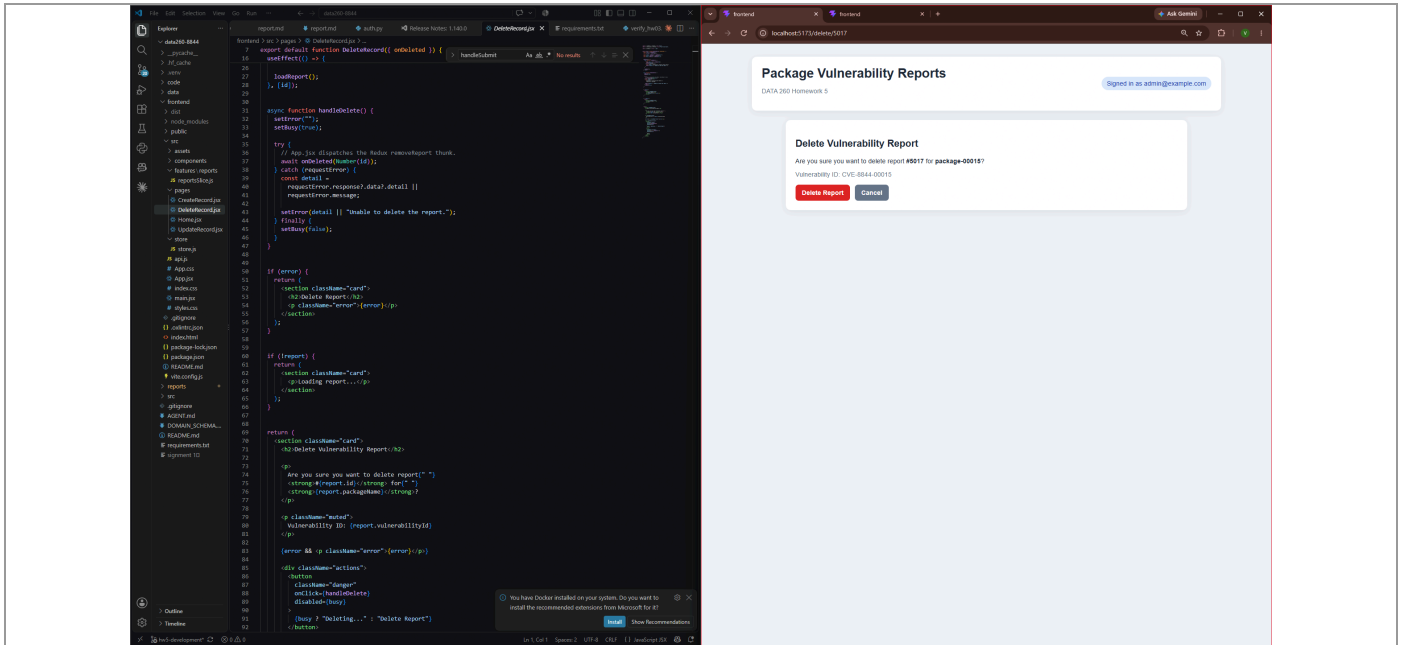


Figure 21: Redux Delete code and UI

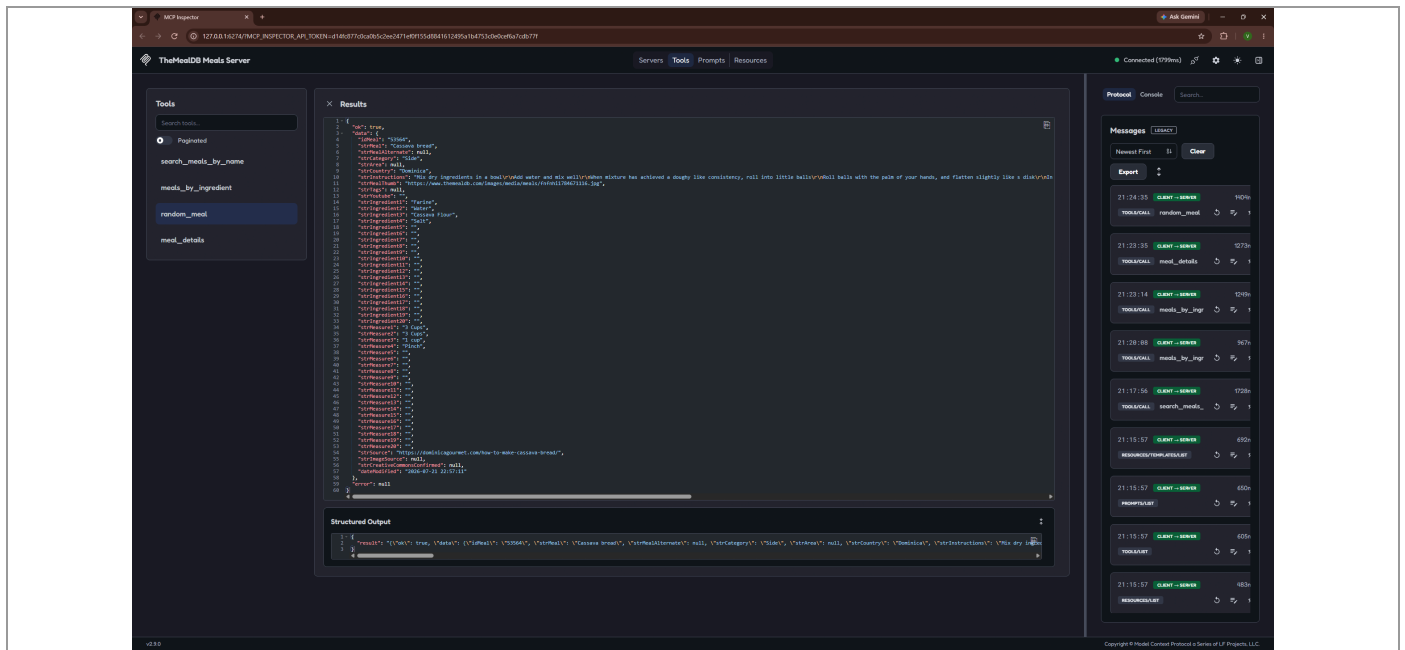


Figure 24: Random meal

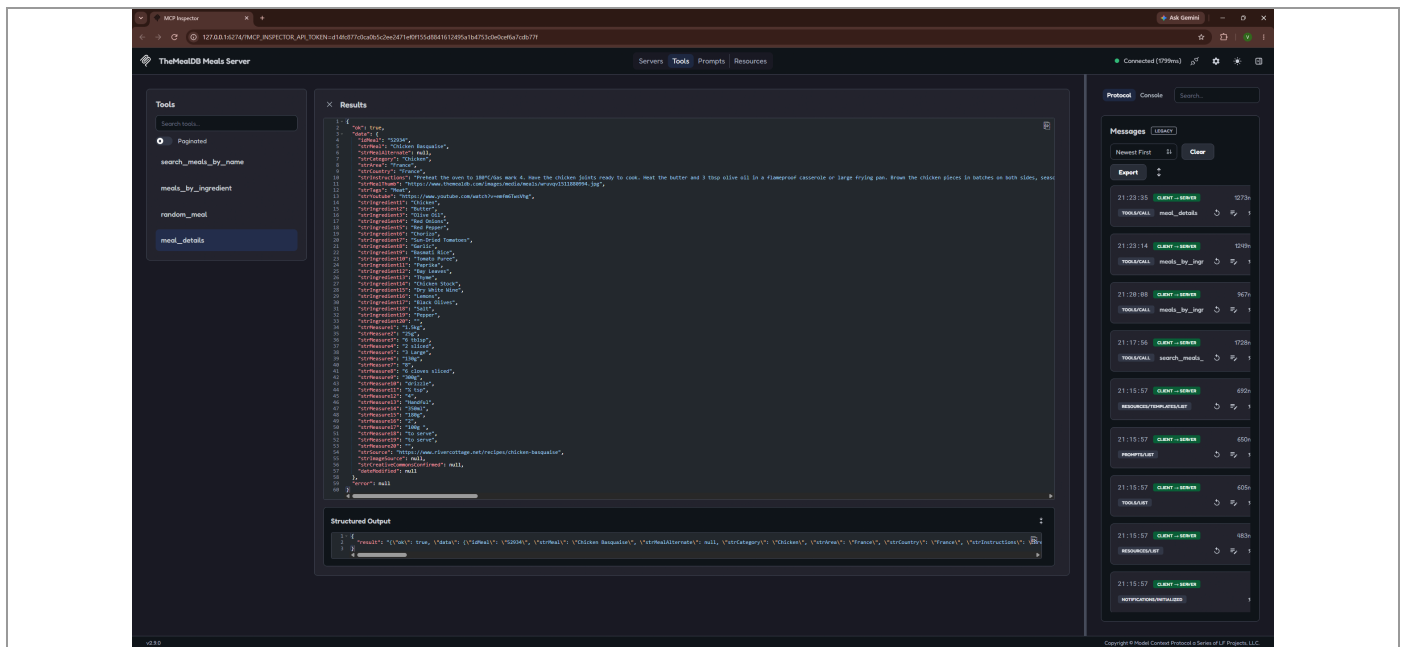


Figure 25: Meal details

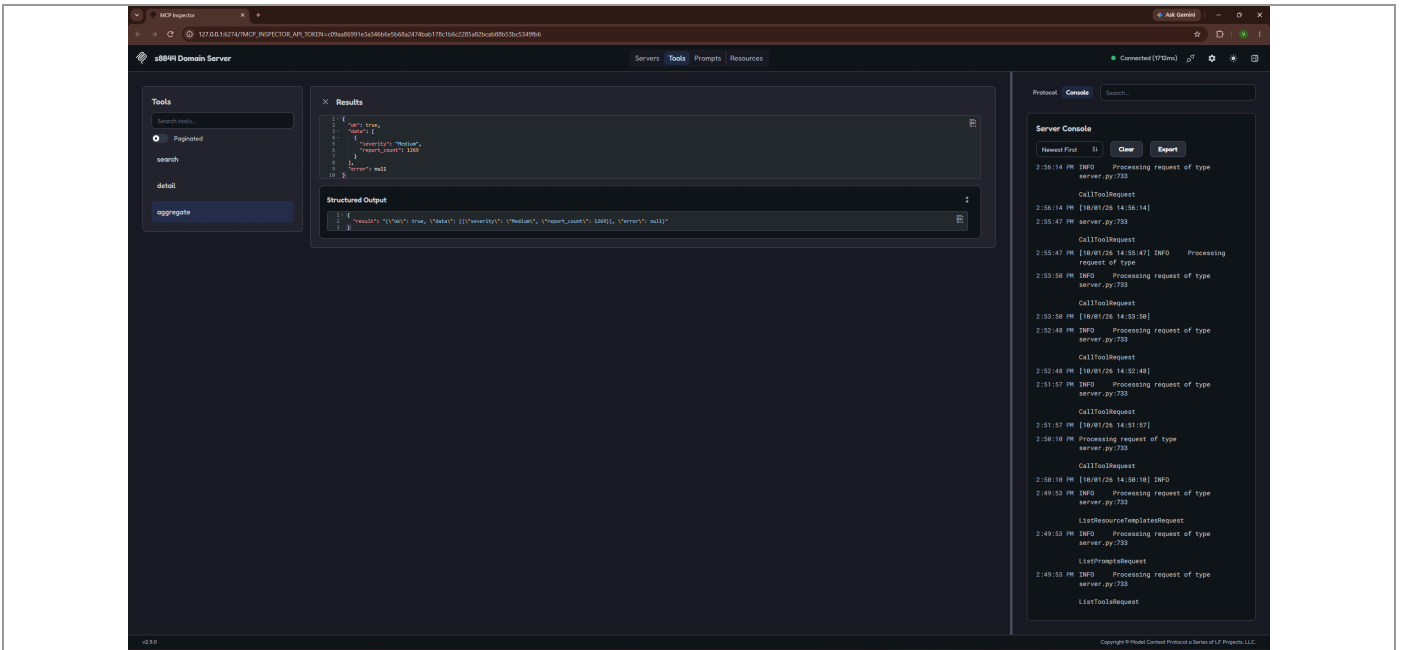


Figure 30: Domain aggregate success

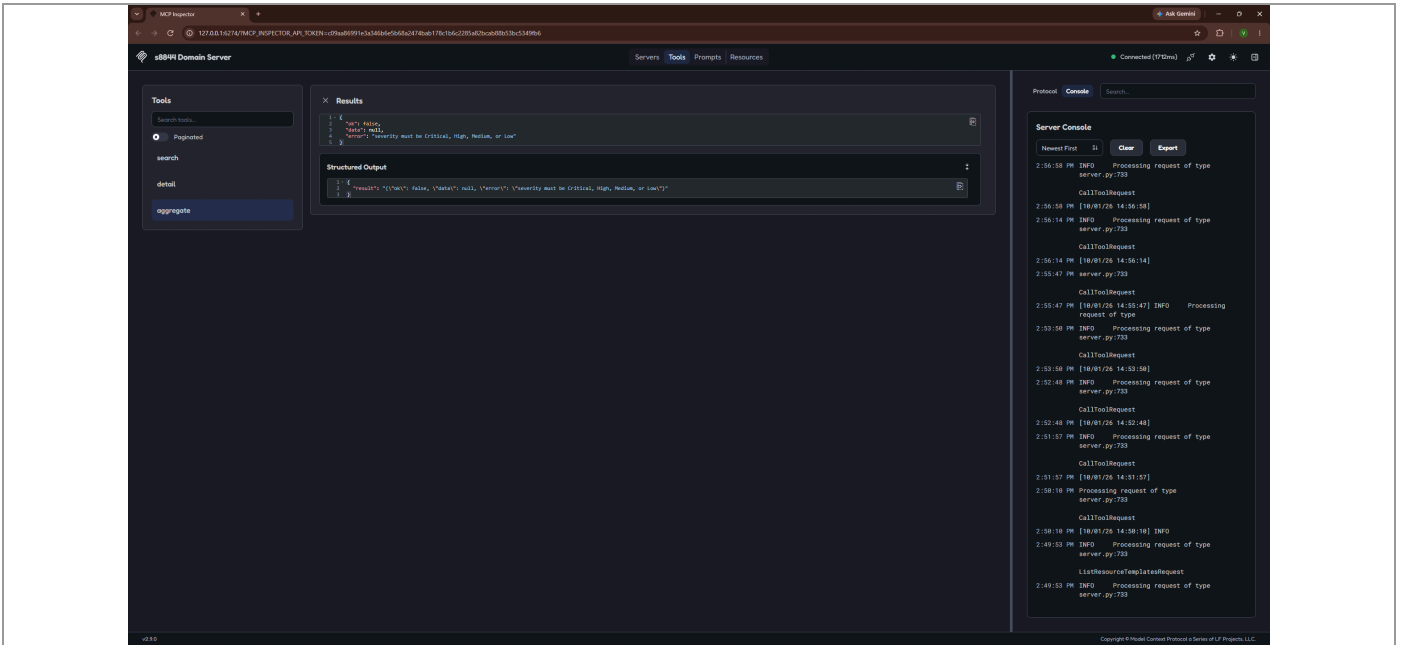


Figure 31: Domain aggregate invalid input

```

PS C:\Projects\data260-8844> cd "C:\Projects\data260-8844"
PS C:\Projects\data260-8844>
PS C:\Projects\data260-8844> & ".\env\Scripts\python.exe" `
> ".\code\mcp\run_fault_injection.py"
etry demonstrations
'demo': 'first_try_success', 'attempts': 1, 'ok': True}
'demo': 'retry_success', 'attempts': 2, 'ok': True}
'demo': 'all_retries_fail', 'attempts': 3, 'ok': False}

ult-injection metrics
'failure_rate': 0.0, "calls": 50, "successes": 50, "success_rate": 1.0, "mean_latency_ms": 1.577, "p99_latency_ms": 1.929, "raw_file": "C:\\Projects\\data
60-8844\\reports\\hw05\\raw\\fault_injection_00.jsonl"}
'failure_rate': 0.2, "calls": 50, "successes": 50, "success_rate": 1.0, "mean_latency_ms": 2.909, "p99_latency_ms": 8.789, "raw_file": "C:\\Projects\\data
60-8844\\reports\\hw05\\raw\\fault_injection_20.jsonl"}
'failure_rate': 0.5, "calls": 50, "successes": 44, "success_rate": 0.88, "mean_latency_ms": 8.842, "p99_latency_ms": 20.776, "raw_file": "C:\\Projects\\da
a260-8844\\reports\\hw05\\raw\\fault_injection_50.jsonl"}

aved raw records to: C:\Projects\data260-8844\reports\hw05\raw
PS C:\Projects\data260-8844> |

```

Figure 32: Retry demonstrations and metrics

```

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\Vrishin Dharmesh KP> cd "C:\Projects\data260-8844"
PS C:\Projects\data260-8844>
PS C:\Projects\data260-8844> & ".\env\Scripts\python.exe" `
> ".\code\mcp\test_execute_tool.py"
PASS search valid
PASS search invalid
PASS detail valid
PASS detail invalid
PASS aggregate valid
PASS aggregate invalid
PASS unknown tool rejected

SUMMARY: 7/7 tests passed
PS C:\Projects\data260-8844> |

```

Figure 33: Offline execute-tool tests

```

PS C:\Projects\data260-8844>
PS C:\Projects\data260-8844> & ".\venv\Scripts\python.exe" `
>> ".\code\mcp\agent.py" --tests
PASS safety allowed call
PASS safety blocked call
PASS MockModel normal completion
PASS MockModel max_steps
SUMMARY: 4/4 agent tests passed
PS C:\Projects\data260-8844> |

```

Figure 34: Agent safety tests

```

Saved metrics to: C:\Projects\data260-8844\reports\hw05\raw\agent_scenarios_metrics.json
Agent runs are logged to:
C:\Projects\data260-8844\reports\hw05\raw\agent_runs.jsonl
PS C:\Projects\data260-8844> Get-Content
>> "C:\Projects\data260-8844\reports\hw05\raw\agent_runs.jsonl" |
>> Select-Object -Last 12
{"user_input": "Find the package.", "step": 2, "action": "final", "final_answer": "The package search completed.", "stop_reason": "model_f",
"final", "trace": [{"step": 1, "tool": "search", "inputs": {"query": "package"}, "result": {"ok": true, "data": [{"name": "fixture-package"}]},
"error": null}]}
{"user_input": "Keep searching.", "step": 2, "action": "final", "final_answer": "The maximum step limit was reached.", "stop_reason": "max",
"steps", "trace": [{"step": 1, "tool": "search", "inputs": {"query": "package"}, "result": {"ok": true, "data": [{"name": "fixture-package"}]},
"error": null}, {"step": 2, "tool": "search", "inputs": {"query": "package"}, "result": {"ok": true, "data": [{"name": "fixture-pack",
age"}]}, "error": null}]}
{"user_input": "Count the vulnerability reports with Medium severity.", "step": 2, "action": "final", "final_answer": "There are 1269 vuln",
erability reports with Medium severity.", "stop_reason": "model_final", "trace": [{"step": 1, "tool": "aggregate", "inputs": {"severity":
"Medium"}, "result": {"ok": true, "data": [{"severity": "Medium", "report_count": 1269}], "error": null}]}
{"user_input": "Count the vulnerability reports with Medium severity.", "step": 1, "action": "final", "final_answer": "Invalid model actio",
n.", "stop_reason": "invalid_model_action", "trace": []}
{"user_input": "Find packages matching package-00001.", "step": 2, "action": "final", "final_answer": "Found package: package-00001 with I",
D 1.", "stop_reason": "model_final", "trace": [{"step": 1, "tool": "search", "inputs": {"query": "package-00001", "limit": 5}, "result": {
"ok": true, "data": [{"id": 1, "name": "package-00001", "ecosystem": "PyPI", "package_code": "s8844-package-00001", "created_at": "2026-01",
-01 00:01:00", "updated_at": "2026-01-01 00:01:00"}]}, "error": null}]}
{"user_input": "Show the details for package ID 1.", "step": 2, "action": "final", "final_answer": "Package details for ID 1:\n\n**ID**:",
1\n\n**Name**:", package-00001\n\n**Ecosystem**:", PyPI\n\n**Package Code**:", s8844-package-00001\n\n**Created At**:", 2026-01-01 00:01:00\n\n **",
Updated At**:", 2026-01-01 00:01:00\n\n**Reports**:",\n1. **Vulnerability ID**:", CVE-2026-8844, **Severity**:", High, **Affected Versions Count**:",
5, **Submission Date**:", 2026-10-01 00:04:11\n2. **Vulnerability ID**:", CVE-2026-8844-TEST-002, **Severity**:", Critical, **Affected Version",
s Count**:", 4, **Submission Date**:", 2026-10-01 00:08:59", "stop_reason": "model_final", "trace": [{"step": 1, "tool": "detail", "inputs": {
"package_id": 1}, "result": {"ok": true, "data": [{"id": 1, "name": "package-00001", "ecosystem": "PyPI", "package_code": "s8844-package-00",
001", "created_at": "2026-01-01 00:01:00", "updated_at": "2026-01-01 00:01:00", "reports": [{"id": 10004, "vulnerability_id": "CVE-2026-88",
44", "severity": "High", "affected_versions_count": 5, "submission_date": "2026-10-01 00:04:11"}, {"id": 10005, "vulnerability_id": "CVE-2",
026-8844-TEST-002", "severity": "Critical", "affected_versions_count": 4, "submission_date": "2026-10-01 00:08:59"}]}, "error": null}]}
{"user_input": "Count the vulnerability reports with Medium severity.", "step": 2, "action": "final", "final_answer": "There are 1269 vuln",
erability reports with Medium severity.", "stop_reason": "model_final", "trace": [{"step": 1, "tool": "aggregate", "inputs": {"severity":
"Medium"}, "result": {"ok": true, "data": [{"severity": "Medium", "report_count": 1269}], "error": null}]}
{"user_input": "Search for password records.", "step": 2, "action": "final", "final_answer": "The search for password records was blocked",
due to a safety rule preventing credential-like searches.", "stop_reason": "model_final", "trace": [{"step": 1, "tool": "search", "inputs":
{"query": "password", "limit": 5}, "result": {"ok": false, "data": null, "error": "safety rule blocked credential-like search"}]}
PS C:\Projects\data260-8844>

```

Figure 35: Ollama scenarios

Submission Files

- RUN_LOG.txt
- METRICS.md
- AI_USE.md
- REFLECTION.md
- verification.json
- raw/agent_runs.jsonl
- raw/agent_scenarios_metrics.json
- raw/fault_injection_00.jsonl
- raw/fault_injection_20.jsonl

- raw/fault_injection_50.jsonl
- raw/fault_injection_metrics.json
- report.pdf